

Software Requirements and Design Document

For

Group <Closet Design Game>

Version 1.0

Authors:

Ethan McConnell
Camryn Morris
Chloe Patrick
Vanessa Campoe
Wesley Jordan
Jessika Tyler

1. Overview (5 points)

Closet Design Game is a web-based single-page application that lets a regular user build and customize outfits on a mannequin. The overall project is intended to support historical and geographically themed clothing templates, interactive drawing tools, mannequin choices, saved outfits, and sharing/export. Increment 1 establishes the basic studio experience: a stable Fabric.js canvas, the studio navigation and filter drawers, an initial clothing catalog structure, garment placement, drawing controls, and clipping so paint stays inside a selected garment. During Increment 1, the interface is built in React with Tailwind CSS and Lucide icons. Users can browse by era and region, choose basic clothing categories, place a garment on the canvas, and customize it with freehand drawing controls for stroke size, color, opacity, and erasing. Garment SVG assets are normalized by a Python preprocessing script so items align consistently with the mannequin/canvas coordinate system.

2. Functional Requirements (10 points)

FR-1 (High): The system shall display a blank, centered design canvas when the studio page loads.

FR-2 (High): The system shall maintain a fixed canvas width and height so the working area remains consistent across screen sizes.

FR-3 (High): The system shall allow the user to draw freehand strokes on the canvas.

FR-4 (High): The system shall allow the user to change the freehand brush stroke width, with values restricted to the allowed range implemented by the application.

FR-5 (High): The system shall allow the user to select the brush color.

FR-6 (High): The system shall allow the user to adjust brush opacity.

FR-7 (High): The system shall provide an eraser mode for removing user-created strokes.

FR-8 (Medium): The system shall provide a slide-out control for browsing or selecting an era.

FR-9 (Medium): The system shall provide a slide-out control for browsing or selecting a geographic region.

FR-10 (High): The system shall provide selectable clothing categories for at least tops, bottoms, and hats in Increment 1.

FR-11 (High): When the user selects an available garment item, the system shall place that garment on the design canvas.

FR-12 (High): A placed garment shall be aligned to the common canvas/mannequin anchor established by the asset pipeline.

FR-13 (High): When the user paints a garment, visible paint strokes shall remain within that garment's clipping boundary and shall not bleed onto the background.

FR-14 (Medium): The clothing catalog data shall identify each garment by its era/decade, geographic region, clothing category, and asset reference so the interface can filter and retrieve items.

FR-15 (Medium): The studio page shall provide a top navigation bar and side-drawer controls needed to access the Increment 1 design functions.

3. Non-functional Requirements (10 points)

NFR-1 - Platform/Portability: The Increment 1 system shall run as a web-based single-page application in modern desktop and mobile browsers; no native installation shall be required.

NFR-2 - Reliability: Reloading the page shall not cause the application or the basic canvas component to crash.

NFR-3 - UI Consistency: The design canvas dimensions shall remain stable even when the surrounding browser viewport changes.

NFR-4 - Responsiveness: Freehand drawing shall remain interactive enough for normal use; drawing frame rate will be measured during Increment 1 testing so lag can be identified.

NFR-5 - Maintainability: UI, canvas behavior, catalog data, and asset-processing responsibilities shall remain separated into clear modules/components so later increments can extend them.

NFR-6 - Asset Consistency: Preprocessed SVG garment assets shall use a common size/alignment convention so equivalent garment types appear in predictable positions when

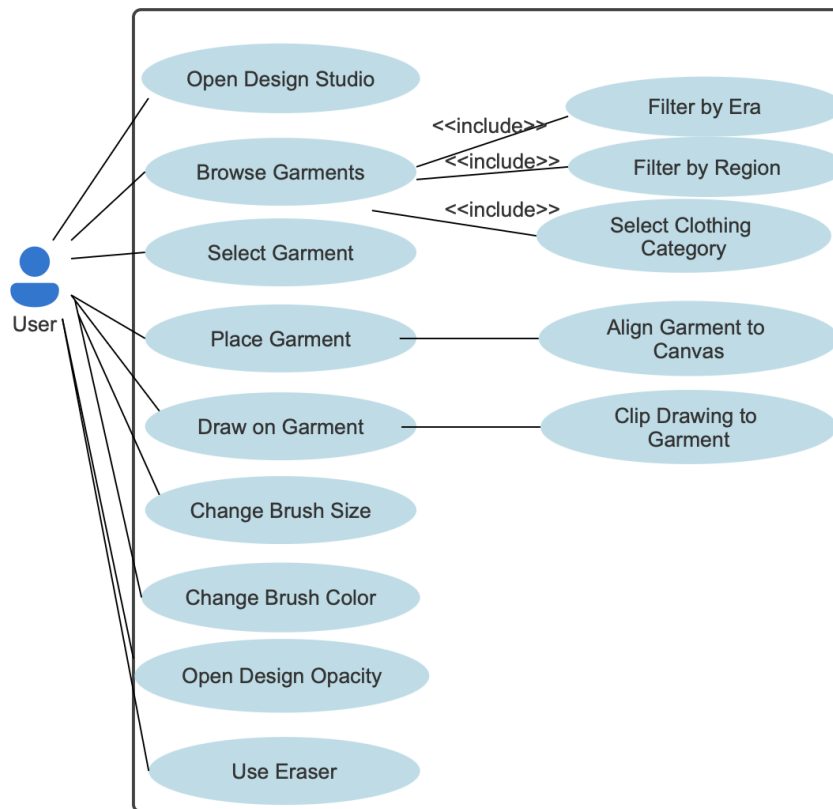
placed.

NFR-7 - Code Quality: Changes merged into the shared repository shall be subject to the team's GitHub review rules requiring review before merge.

4. Use Case Diagram (10 points)

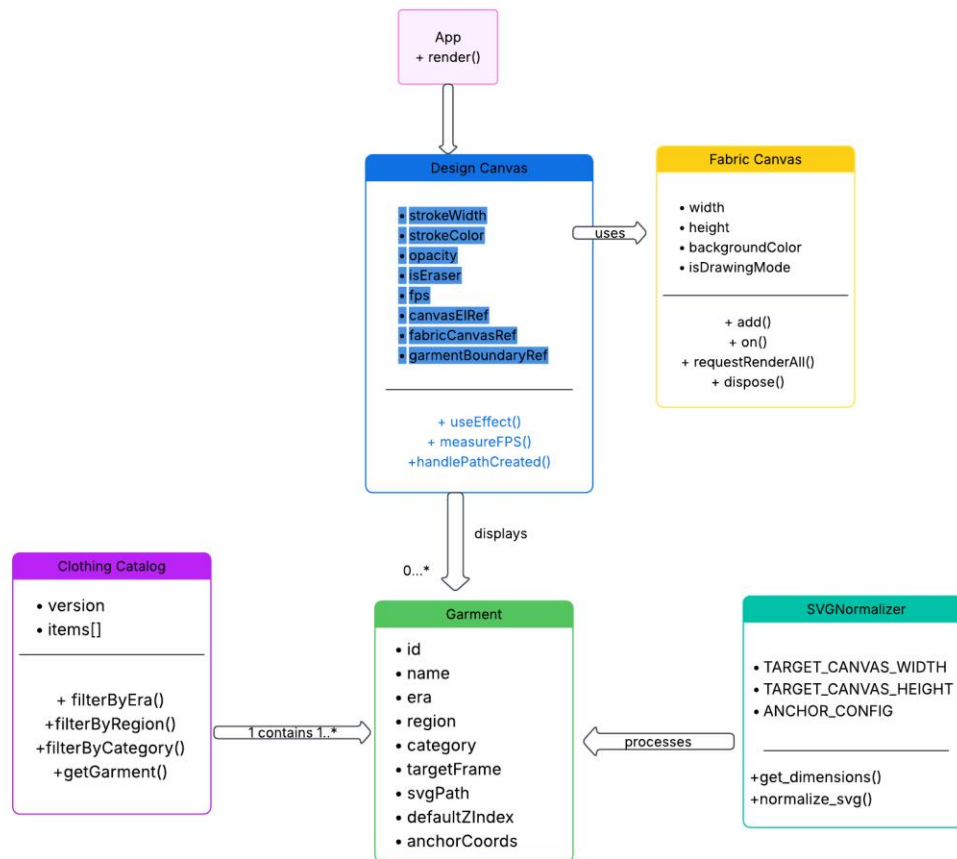
The following use case diagram presents the primary interactions between a user and the Closet Design Game during Increment.

The user can access the design studio, browse and filter available garments, select and place garments on the design canvas, and customize garments using the available drawing tools. Drawing controls include brush size, color, opacity, and an eraser. The system also aligns garment assets to the shared canvas coordinate system and clips freehand drawing to the selected garment boundary.



5. Class Diagram and/or Sequence Diagrams (15 points)

Class Diagram:



6. Operating Environment (5 points)

Closet Design Game operates as a browser-based single-page application for desktop and mobile browsers. The Increment 1 client is built with JavaScript/TypeScript, HTML, and CSS. React provides the component-based UI and state structure; Vite provides frontend development/build tooling; Tailwind CSS is used for the studio layout; Lucide provides interface icons; and Fabric.js provides the interactive canvas, drawing tools, object placement, and clipping behavior. Garment and mannequin art is represented with SVG assets. Development requires a computer capable of running a modern web browser and the Node.js/npm toolchain needed by the React/Vite project. The source is maintained in a shared GitHub repository with review rules. A separate Python script is used during development to resize and normalize incoming SVG garment assets before those files are saved in the application's assets folder. No server-side database or user account system is required for Increment 1.

7. Assumptions and Dependencies (5 points)

A-1: The browsers used to run the application support the HTML5 canvas and SVG features required by Fabric.js.

A-2: React, Vite, Fabric.js, Tailwind CSS, and Lucide remain compatible with the versions selected by the team during Increment 1.

A-3: Garment source SVG files are valid enough for the Python normalization script to resize and realign them without manual reconstruction.

A-4: The team can define a consistent canvas/mannequin coordinate system that all processed garments can share.

A-5: The Increment 1 catalog can be represented as static JSON and does not require a remote

database.

A-6: GitHub and the team repository remain available for collaboration and review-before-merge workflow.

A-7: Features planned for later increments - mannequin switching, save/load storage, undo/redo, gallery, PNG export, and Web Share integration - are outside the Increment 1 requirement baseline.